

ЗВІТ ДО ПРОЄКТУ

«Tree-Based Database Engine»

Навчальна база даних на основі шести різних дерев пошуку

Команда:

- Зайць Поліна
- Жванко Ярина
- Волоський Віталій
- Двібородчин Андрій
- Ягода Микита (ментор)

Мова реалізації: Python 3

Зовнішні бібліотеки: matplotlib, Pillow

Вбудовані модулі: sqlite3, argparse, json, random, time, statistics, gc

1 ВСТУП ТА МЕТА ПРОЄКТУ

У межах курсу ми реалізували невелику навчальну систему керування базою даних, у якій усі записи зберігаються в одному з шести самобалансованих дерев пошуку. Користувач сам обирає, яке саме дерево використовувати, вводять у консолі команди мовою, схожою на SQL (INSERT, SELECT, UPDATE, DELETE), а програма паралельно зберігає кадр-знімок дерева після кожної операції. Із цих кадрів автоматично збирається анімований GIF, який наочно показує, як саме дерево перебудовується під час кожної дії. Окремим скриптом-бенчмарком ми порівняли наші реалізації між собою та з повноцінною реляційною СУБД SQLite на однакових сценаріях - це дозволило кількісно оцінити сильні й слабкі сторони кожної структури не лише в теорії, а й у практичних умовах інтерпретованого Python.

Головна мета проєкту:

1. На практиці закріпити роботу самобалансованих дерев - від простих обертань в AVL до балансування «ширшими» B-деревами.
2. Побудувати єдиний інтерфейс (ADT), щоб з точки зору бази даних усі дерева були взаємозамінними.
3. Візуально продемонструвати, як саме структура змінюється після кожної операції - для самих себе та для презентації.
4. Виміряти й порівняти продуктивність - щоб бачити, де теорія збігається з практикою, а де поведінка диктується деталями реалізації, кешем процесора та накладними витратами Python.

2 ЗАГАЛЬНА АРХІТЕКТУРА

Проект побудований на чіткому розділенні відповідальності - кожен модуль робить свою одну справу:

- `main.py` - точка входу для інтерактивної бази даних. Парсить аргументи, створює потрібне дерево, запускає цикл "читай-виконуй-друкуй-зберігай" і наприкінці склеює GIF із усіх знімків.
- `benchmark.py` - окрема програма, яка автоматично проганяє всі дерева на однакових даних, заміряє час, будує графіки в темній темі та зберігає CSV-таблиці.
- `database.py` - клас `Record` (запис із ключем та значенням) і клас `DatabaseEngine`, який вміє розбирати SQL-подібні команди у звичайному рядку. Сам не знає, яке під ним дерево - спілкується через єдиний інтерфейс.
- `tree_adt.py` - абстрактний клас `Tree_ADT`. Описує контракт, якому має відповідати будь-яке наше дерево: `add`, `delete`, `find`, `inorder/preorder/postorder`, `_rotate_left`, `_rotate_right`.
- `avl_tree.py` - AVL-дерево.
- `splay_tree.py` - Splay-дерево.
- `rb_tree.py` - Червоно-чорне дерево (з NIL-сентинелем).
- `b_tree.py` - B-дерево довільного порядку m .
- `b_plus_tree.py` - B+ дерево з горизонтальними зв'язками між листами.
- `twothree_tree.py` - 2-3 дерево як окремий випадок B-дерева з $m = 3$.
- `visual.py` - система візуалізації. Один базовий клас `Visualizer` для бінарних дерев і окремий `BTreeVisualizer` для дерев із багатоключовими вузлами; підкласи додають специфіку (кольори RB, balance factor у AVL, пунктирні зв'язки між листами B+ дерева).
- `requirements.txt` - лише дві справжні залежності: `matplotlib` і `Pillow`. Усе інше - це стандартна бібліотека Python.

Ключова ідея архітектури - поліморфізм через ADT. Клас `DatabaseEngine` не знає про конкретні дерева взагалі: він отримує об'єкт, який уміє `add` / `find` / `delete`, і всі наші реалізації цей контракт виконують. Завдяки цьому переключення між AVL, RB чи B+ під час запуску зводиться до однієї опції командного рядка `-tree`.

3 РЕАЛІЗОВАНІ СТРУКТУРИ ДАНИХ

Усі шість дерев успадковуються від абстрактного `Tree_ADT`, що гарантує єдиний інтерфейс і дозволяє використовувати їх у `DatabaseEngine` взаємозамінно. Нижче - короткий опис ідеї та особливостей реалізації.

3.1 AVL-дерево (avl_tree.py)

Класичне самобалансоване BST: для кожного вузла різниця висот лівого й правого піддерев не перевищує 1. Після кожної вставки та видалення ми піднімаємось рекурсією до кореня, оновлюємо висоту вузла і за потреби виконуємо одне з чотирьох обертань:

- LL: одне праве обертання;
- RR: одне ліве обертання;
- LR: ліве у сина, праве у батька;
- RL: праве у сина, ліве у батька.

Гарантована висота - $\mathcal{O}(\log n)$, тому всі три основні операції (пошук, вставка, видалення) виконуються за $\mathcal{O}(\log n)$ у найгіршому випадку. Після кожного обертання викликається `save_tree_snapshot`, тому в GIF видно навіть проміжні стани балансування.

3.2 Splay-дерево (splay_tree.py)

Бінарне дерево пошуку без явного інваріанту висоти, але з принципом "щойно знайшли - підняли до кореня". Для цього виконуються повороти `zig` / `zig-zig` / `zig-zag`, які за $\mathcal{O}(\log n)$ АМОРТИЗОВАНО переміщують останній доступаний елемент у корінь. Це дає чудову поведінку, коли ми часто звертаємось до невеликого "гарячого" набору ключів, але в жанрі рівномірно-випадкового доступу splay-дерево повільніше за AVL/RB через постійні перевитрати на сплеювання навіть простих SELECT-ів. У нашій реалізації `find` теж сплеює знайдений вузол, тому навіть "чисто читальний" сценарій модифікує дерево - це класична поведінка splay-дерева, і саме вона визначає його профіль у тестах.

Окремо у файлі є функція `validate_tree_integrity`, що перевіряє:

- чи відсортований `inorder`-обхід;
- чи немає дублікатів;
- чи коректні `parent`-посилання у вузлів.

Цю функцію ми використовували під час розробки для відлову помилок.

3.3 Червоно-чорне дерево (rb_tree.py)

Класичний підхід Cormen et al. ("Introduction to Algorithms"):

- кожен вузол має колір R або B;
- усі листи - спільний чорний "сентинел" `_nil`;
- корінь чорний, червоні вузли не мають червоних дітей;
- на будь-якому шляху від кореня до листів - однакова кількість чорних вузлів.

У нас є повний набір процедур:

- `_rotate_left` / `_rotate_right`;
- `_transplant` - заміна піддерева;
- `_insert_fixup` із розбором `zig-zig` / `zig-zag` (LL, LR, RR, RL);

- `_delete_fixup` для випадку видалення чорного вузла - балансування подвійно-чорного через брата `w`.

Спеціально для звітності і відлагодження в `_insert_fixup` і `_delete_fixup` усі гілки розбиті на іменовані допоміжні методи (`_insert_fixup_zig_zig_left` тощо) - стало значно зрозуміліше, ніж монолітний `while`.

3.4 2-3 дерево (`twothree_tree.py`)

Кожен внутрішній вузол має 2 або 3 дітей і 1 або 2 ключі. Ми отримали його "майже безкоштовно" 2-3 дерево є окремим випадком В-дерева з порядком $m = 3$, і клас `TwoThreeTree` успадковується від `BTree(m=3)` без додаткового коду. Це наочна ілюстрація узагальнення: одна реалізація - два дерева.

3.5 В-дерево (`b_tree.py`)

В-дерево порядку m : вузол містить до $m - 1$ ключів і має до m дітей, усі листи на одному рівні, ключі в межах вузла відсортовані. У реалізації:

- вставка йде "знизу вгору": якщо лист переповнюється, ключ-медіана "піднімається" до батька разом із новим правим вузлом;
- якщо переповнюється корінь - створюється новий корінь, тому висота зростає тільки одночасно з усім деревом;
- видалення - повна класична схема:
 - предок (max лівого піддерева) або наступник (min правого);
 - запозичення в сусідньої сторінки (`_borrow_prev` / `_borrow_next`);
 - злиття двох сусідів через ключ батька (`_merge`);
 - протягування "underflow" вгору рекурсією.

Завдяки порядку $m = 4$ за замовчуванням дерево виходить дуже плоске - це менше довгих ланцюжків посилань і краща поведінка кешу, що добре видно в результатах бенчмарку.

3.6 В+ дерево (`b_plus_tree.py`)

Структура, на якій базуються індекси у MySQL/InnoDB та інших СУБД. Відмінності від В-дерева:

- реальні дані зберігаються тільки в листах;
- внутрішні вузли містять лише ключі-роздільники для навігації;
- усі листи зв'язані горизонтальними посиланнями `next`, що робить діапазонні запити та послідовний обхід дуже швидкими (один прохід по списку, без рекурсії).

У нашій реалізації:

- `_split_leaf` копіює медіанний ключ у внутрішній вузол (а не "виштовхує" як у звичайному В-дереві);
- `_split_internal` працює як у класичному В-дереві;

- `find` опускається в лист і перевіряє ключі лінійно - бо в B+ дереві внутрішні ключі є лише дороговказами, а не справжніми записами;
- `inorder` обхід реалізований через посилання `next`, без рекурсії.

Візуально у GIF-кадрах B+ дерева пунктирні стрілки між листами показують ці `next`-посилання - це одразу пояснює, чому B+ так популярне для дискових індексів.

3.7 Спільний інтерфейс

Усі шість дерев надають однаковий контракт: `add(item)`, `delete(item)`, `find(value)`, `inorder/preorder/postorder`. Завдяки цьому:

- `DatabaseEngine` не залежить від конкретного дерева;
- `benchmark.py` обгортає кожне дерево в `TreeEngine` з єдиним API `setup` → `insert/select/update` → `teardown`;
- SQLite підключається через таку саму обгортку (`SQLiteEngine`), і весь бенчмарк не помічає різниці між реальною СУБД і нашими деревами.

4 БАЗА ДАНИХ І ОБРОБКА КОМАНД

Файл `database.py` містить два класи:

- `Record` - простий контейнер "ключ + значення із переважаними операторами порівняння (`__lt__`, `__gt__`, `__eq__` тощо), щоб дерева могли впорядковувати записи прямо за об'єктами. Ключ - ціле число, значення - будь-який Python-об'єкт (зазвичай `dict`, отриманий з JSON).
- `DatabaseEngine` - мозок інтерпретатора команд. Він:
 - бере "сирий" текстовий рядок;
 - розбиває на команду / `id` / дані;
 - намагається спарсити дані як JSON, а в разі невдачі залишає рядком;
 - звертається до `tree.find` / `tree.add` / `tree.delete` за єдиним ADT-інтерфейсом;
 - повертає текстовий результат у стилі "OK: ..." / "Error: ...".

Підтримуються чотири команди:

- **INSERT 101 {"name": "Mykyta" "faculty": "CS"}** - Додає новий запис. Якщо `id` вже існує - повертає помилку й пропонує використати `UPDATE`.
- **SELECT 101** - Шукає запис за `id`. Повертає значення або повідомлення про відсутність.
- **UPDATE 101 {"name": "Anna"}** - Знаходить запис і замінює `value` на місці. Дерево не перебудовується (ключ не змінився).
- **DELETE 101** - Видаляє запис. Викликає `tree.delete` і знімок дерева.
- **q** - Завершити сесію. Перед виходом програма склеює всі знімки, зроблені під час роботи, у GIF-файл `results/GIFs_with_database/<tree>_db_animation.gif`.

Перед кожною мутуючою операцією `database.py` викликає:

```
visual.set_label(f"INSERT: {record_id}")
self.tree.add(new_record)
visual.save_tree_snapshot(self.tree)
```

Тобто візуалізатор отримує опис кроку і знімок одразу після нього - саме це й робить підсумковий GIF читабельним і "розповідним".

5 СИСТЕМА ВІЗУАЛІЗАЦІЇ

`visual.py` відповідає за роботу з `matplotlib` і дав решті коду один зручний інтерфейс із трьох функцій:

- `set_label(text)` - встановити "заголовок" наступного кадру.
- `save_tree_snapshot(tree)` - відмалювати поточний стан дерева у PNG.
- `make_gif(folder, output)` - склеїти всі PNG у GIF.

Реалізовано п'ять класів-візуалізаторів:

- **Visualizer** - базовий, для бінарних дерев. Сам обчислює координати вузлів рекурсією, малює круглі вузли, ребра, заголовок-каунтер кадру.
- **AVLVisualizer** - успадковує **Visualizer**, додає balance factor біля кожного вузла і змінює колір залежно від BF (синій - нуль, зелений - ± 1 , жовтий - переповнення).
- **RBVisualizer** - успадковує **Visualizer**, замінює кольори на класичні RED / BLACK і знає про NIL-сентинель.
- **BTreeVisualizer** - окрема реалізація для B-дерева: вузли - це прямокутники з кількома ключами всередині, між ними - вертикальні лінії-роздільники.
- **BPlusVisualizer** - успадковує **BTreeVisualizer**, забарвлює листові вузли в зелений і ще домальовує пунктирні стрілки `next` між листами.

Усе оформлене в єдиній темній палітрі (фон #111318, акценти синій і зелений), щоб GIF-и виглядали стилістично однорідно - і це помітна перевага: без спільної естетики звіт виглядав би "клаптиково".

Технічні деталі, які варто зазначити:

- `matplotlib` працює в режимі Agg (без GUI), тому код не вимагає графічного середовища і однаково запускається на сервері й локально;
- PIL використовується тільки на фінальному кроці - щоб склеїти PNG у GIF із заданою тривалістю кадрів і подовженим останнім кадром (`last_ms=3000`), який дає ока секунду-дві "видихнути" перед перезапуском анімації;
- кадри зберігаються в тимчасову папку `frames/`, яка очищається на старті `main.py` - щоб результат не плутався з попередніми сесіями.

6 БЕНЧМАРК (benchmark.py)

Він робить порівняння наших шести дерев між собою та з SQLite.

6.1 Що саме ми міряємо

Для заданого N будуються однакові набори ключів:

- `keys_insert` - перестановка чисел $[0..N)$ (порядок випадковий);
- `keys_select` - N випадкових вибірок із наявних ключів;
- `keys_update` - те саме, але для UPDATE;
- `keys_delete` - копія `keys_insert` у новому випадковому порядку.

Усі двигуни на одному N бачать однієї й ті самі ключі - це робить порівняння чесним: випадковість одна, а не нова для кожного двигуна.

6.2 Як саме виконується вимірювання

- Перед кожним прогоном викликається `engine.setup()`;
- Потім відключається збирач сміття (`gc.disable()`), щоб непередбачуваний "stop-the-world" не зіпсував результат;
- Час замірюється `time.perf_counter()` - найточніший таймер Python;
- Перетягування з пам'яті в пам'ять (для SQLite - `:memory:`, `PRAGMA journal_mode = MEMORY`, `PRAGMA synchronous = OFF`), щоб порівнювати швидкість самих структур, а не дискового I/O. Інакше SQLite фактично змагався б із власною файловою підсистемою, а не з нашими деревами.
- Після прогону `engine.teardown()` звільняє посилання і примусово запускає `gc.collect()` - наступний двигун стартує "з чистими руками".

6.3 Стабільність результатів

- `-repeats N` - кожен сценарій можна повторити N разів, підсумкове значення береться як МЕДІАНА. Це краще за середнє, бо нечутливе до випадкових "виплесків" (раптовий прокид ОС, міграція потоку між ядрами тощо).
- `-seed` - фіксує випадковість для відтворюваності.
- `-quick` - швидкий прогон на малих N (200, 500, 1000, 2000) із `-repeats 2` - корисно перед "великим" запуском, щоб переконатися, що нічого не зламалося.

6.4 Що зберігається на виході

Усе складається в каталог `results/` (можна змінити через `-output`):

- `results/raw_results.csv` - повні "сирі" дані: для кожного двигуна, кожного N і кожної операції - загальний час, час на одну операцію та пропускну здатність.
- `results/tables/summary_<op>.csv` - зведена таблиця по кожній операції: рядки - структури, стовпці - розміри N .
- `results/plots/lines_all.png` - сітка 2×2 з усіма чотирма операціями і всіма двигунами на одному графіку.

- `results/plots/lines_<op>.png` - окремий лінійний графік для кожної операції (час на одну операцію проти N).
- `results/plots/bars_<op>.png` - стовпчикова діаграма для найбільшого N: відсортована від найшвидшого до найповільнішого, з підписами точних значень.

Ще під час самого запуску в консоль друкуються ASCII-таблиці, щоб результат можна було швидко прочитати, не відкриваючи CSV.

7 РЕЗУЛЬТАТИ ВИМІРЮВАНЬ

Остання збережена перевірка на $N = 100\,000$ та $N = 150\,000$ записів. Нижче зведено час однієї операції в мікросекундах (мкс/оп).

7.1 INSERT (мкс/оп)

Структура	N=100000	N=150000
SQLite	4.8	6.2
RB	8.9	8.9
B-дерево	9.4	10.9
B+ дерево	10.2	11.3
2-3 дерево	13.0	13.1
AVL	15.8	17.4
Splay	21.1	21.3

7.2 SELECT (мкс/оп)

Структура	N=100000	N=150000
SQLite	2.7	3.1
AVL	5.7	5.9
RB	9.2	7.9
B+ дерево	8.4	8.7
B-дерево	9.8	9.8
2-3 дерево	12.0	10.9
Splay	23.3	22.3

7.3 UPDATE (мкс/оп)

Структура	N=100000	N=150000
SQLite	4.3	4.9
AVL	6.3	6.5
RB	9.5	8.0
B+ дерево	9.5	9.1
B-дерево	10.3	11.7
2-3 дерево	12.3	11.9
Splay	23.4	24.2

7.4 DELETE (мкс/оп)

Структура	N=100000	N=150000
SQLite	3.4	4.1
B+ дерево	10.4	11.7
RB	10.6	9.1
B-дерево	13.0	14.1
2-3 дерево	13.4	15.8
AVL	23.3	21.2
Splay	25.4	25.3

8 АНАЛІЗ І ВИСНОВКИ

8.1 SQLite - переможець на кожній операції

SQLite виграє у всіх чотирьох сценаріях, причому з відривом у 2-5 разів. Це абсолютно очікувано: SQLite написано на C, у ньому оптимізована робота з пам'яттю, query planner, кешування сторінок, а за нашими PRAGMA-налаштуваннями він взагалі працює повністю в RAM. Наші дерева - на чистому Python: кожне порівняння `Record < Record` це виклик `__lt__`, кожен прохід - Python-фрейм, інтерпретатор, динамічна типізація. Тільки це додає накладних витрат.

Висновок: ми не намагалися "перемогти" SQLite - ми зрозуміли, чому саме він швидкий. І в цьому SQLite виконує роль "опорної точки": показує, наскільки далеко наші чисто-навчальні реалізації від промислового рівня.

8.2 RB-дерево - найшвидше серед наших

RB-дерево стабільно найшвидше у трьох операціях із чотирьох (INSERT, SELECT і DELETE на N=150000) і друге в UPDATE. Причини:

- менша кількість обертань на операцію, ніж у AVL, бо RB допускає "слабший" інваріант (різниця висот до $2\times$ замість $1.44\times$);
- відсутність повторних обчислень `height` у вузлах - лише зміна кольору;
- саме видалення в RB набагато простіше, ніж у AVL: класичний `_delete_fixup`, як виявилося, працює стабільніше за наш рекурсивний `AVL.delete` із `balance factor`.

8.3 B-дерево та B+ дерево - щільні й кешолюбні

B-дерево і B+ дерево показують дуже близьку до RB продуктивність і часто навіть випереджають його в DELETE. Причина - низька висота: при $m = 4$ висота дерева на 100 000 елементах усього 8-9 рівнів, тоді як RB іноді має 30+. Менше посилань = менше random-доступу по пам'яті = краща локальність даних. B+ дерево додатково виграє в DELETE: оскільки реальні значення лежать тільки в листах, а внутрішні ключі - це лише дороговкази, видалення не вимагає шукати наступник у іншому піддереві. Це видно і в графіку: на видаленні B+ обходить навіть RB.

8.4 AVL - найкраще на читанні

На SELECT і UPDATE AVL випереджає всі інші наші дерева (5.7 і 6.3 мкс/оп проти 8-12 у решти). Це теж очікувано: завдяки жорсткішому інваріанту в AVL мінімальна можлива висота, тому пошук - найшвидший. Натомість на INSERT і особливо на DELETE AVL програє: жорсткий інваріант коштує більше обертань, ніж у RB. Висновок: AVL - оптимальний вибір, коли читань значно більше, ніж записів. У реальній СУБД з типовим співвідношенням 80/20 це поширений сценарій.

8.5 Splay-дерево - стабільно найповільніше

Splay-дерево - найповільніше з нашого набору на BCIX чотирьох операціях. На перший погляд це виглядає несподівано, адже splay має дуже хороші амортизовані оцінки. Але є нюанс:

- амортизована оцінка $\mathcal{O}(\log n)$ спрацьовує лише на сценаріях із повторним доступом до невеликого "гарячого" набору;
- наш бенчмарк навмисне використовує рівномірно випадковий доступ - кожен ключ рівноймовірний;
- на такому профілі splay фактично платить за сплеювання без жодної компенсаційної переваги;
- навіть `find()` сплеює дерево, тобто SELECT теж модифікує структуру і триває довше за всі інші дерева.

Висновок: splay не "поганий у нього просто інша ніша. На сценаріях типу "20% ключів отримують 80% звертань" він би виявився конкурентним.

8.6 2-3 дерево

2-3 дерево показує "середні" результати - між RB та splay. Це логічно, бо $m = 3$ - найменший можливий порядок B-дерева, тому й переваги "плоскої структури" мінімальні. У навчальному плані цінне інше: ми отримали повноцінне 2-3 дерево простим успадкуванням від `BTree(m=3)`, без жодного нового рядка логіки.

8.7 Загальна асимптотика

Якщо подивитись на пари ($N=100000$, $N=150000$): час зростає приблизно лінійно за N . Ділимо на N - отримуємо мкс/оп майже сталі (з невеликим зростанням). Це експериментально підтверджує теоретичну оцінку $\mathcal{O}(\log n)$ на операцію: загальний час N операцій - це $N \cdot \log n$, тобто майже лінія, з логарифмічно повільним нахилом, який неможливо побачити неозброєним оком між 100k і 150k.

9 ЩО МИ З ЦЬОГО ВИНЕСЛИ

1. Один абстрактний інтерфейс є важливим. Завдяки `Tree_ADT` ми за кілька хвилин підставили SQLite поруч з нашими деревами без жодних змін у бенчмарку.
2. Візуалізація - потужний навчальний інструмент. Дивитися, як AVL "розкручує" LR-обертання чи як B+ дерево розщеплює лист і протягує ключ-розділювач, у тисячу разів корисніше, ніж читати про це в книжці.

3. У Python важко змагатися з C - і це нормально. Нашою метою було вивчити структури, а не побити SQLite. Цифри 8-25 мкс/оп у чистому Python для самобалансованих дерев - це гарний результат.
4. Бенчмарк потрібно проєктувати акуратно: однакові ключі для всіх двигунів, відключений GC, медіана з кількох прогонів, фіксований seed.
5. Кожне дерево має свою нішу.
6. Найбільш практично корисне у "великому світі" є B+ дерево. Саме його структура лежить в основі індексів реальних СУБД, і тепер ми точно розуміємо чому: листи, зв'язані next, плюс плоска структура з високим коефіцієнтом розгалуження.

10 ЯК ЗАПУСТИТИ ПРОЄКТ

10.1 Встановлення

```
git clone https://github.com/rini1ko/comp_proj
cd comp_proj
pip install -r requirements.txt
```

10.2 Інтерактивна база даних

```
python main.py --tree avl
```

Замість avl можна вказати: splay, rb, btree, bplus, 2-3. Команди в самій програмі:

```
INSERT 101 {"name": "Mykyta"}
SELECT 101
UPDATE 101 {"name": "Anna"}
DELETE 101
q
```

Після виходу зберігається GIF у results/GIFs_with_database/.

10.3 Бенчмарк

Швидка перевірка:

```
python benchmark.py --quick
```

Повноцінний прогон зі стандартними N (1000...50000):

```
python benchmark.py
```

Власні розміри і кілька повторів (для медіани):

```
python benchmark.py --sizes 1000 5000 10000 25000 --repeats 3
```

Корисні прапорці:

--no-splay	не тестувати Splay
--no-sqlite	не тестувати SQLite
--btree-m 4	порядок B-дерева
--bplus-m 4	порядок B+ дерева
--output my_run	куди складати результати
--seed 42	зерно для випадковості

11 ПРИКЛАДИ ЗАПУСКУ І ВИВІД ПРОГРАМИ

У цьому розділі - конкретні сесії, які ми реально проганяли під час розробки та підготовки звіту. Усе нижче - справжні команди та очікувані відповіді програми, скопійовані з консолі.

11.1 Приклад сесії з AVL-деревом

```
> python main.py --tree avl
Database started (Tree: AVL).
Example commands:
  INSERT 101 {"name": "Mykyta", "faculty": "CS"}
  SELECT 101
  UPDATE 101 {"name": "Mykyta", "faculty": "DA"}
  DELETE 101
Enter 'q' to quit.

> INSERT 101 {"name": "Mykyta", "faculty": "CS"}
OK: Record added
> INSERT 50 {"name": "Anna", "faculty": "AM"}
OK: Record added
> INSERT 200 {"name": "Olha", "faculty": "FL"}
OK: Record added
> INSERT 25 {"name": "Petro"}
OK: Record added
> INSERT 75 {"name": "Iryna"}
OK: Record added
> SELECT 101
OK: Found {'name': 'Mykyta', 'faculty': 'CS'}
> UPDATE 101 {"name": "Mykyta", "faculty": "DA"}
OK: Record 101 updated
> SELECT 101
OK: Found {'name': 'Mykyta', 'faculty': 'DA'}
> DELETE 50
OK: Record 50 deleted
> SELECT 50
Error: Record not found
> q
Exiting database...
```

Після виходу автоматично створюється файл-анімація: `results/GIFs_with_database/avl_db_animation.gif`. У ньому покадрово видно, як після INSERT 25 і INSERT 75 у дереві відбувається обертання навколо вузла 50 (LL-випадок), а після DELETE 50 - повторне балансування.

11.2 Приклад сесії з B+ деревом

```
> python main.py --tree bplus
Database started (Tree: BPLUS).
...
> INSERT 10 {"city": "Lviv"}
OK: Record added
> INSERT 20 {"city": "Kyiv"}
```

```

OK: Record added
> INSERT 30 {"city": "Odesa"}
OK: Record added
> INSERT 40 {"city": "Kharkiv"}
OK: Record added
> INSERT 50 {"city": "Dnipro"}
OK: Record added
> INSERT 25 {"city": "Lutsk"}
OK: Record added
> SELECT 30
OK: Found {'city': 'Odesa'}
> DELETE 20
OK: Record 20 deleted
> q
Exiting database...

```

У збереженому GIF `results/GIFs_with_database/bplus_db_animation.gif` особливо добре видно зелені листові вузли та пунктирні `next`-стрілки між ними - саме та структура, яку ми описували в розділі 3.6.

11.3 Приклад сесії зі Splay-деревом

```

> python main.py --tree splay
Database started (Tree: SPLAY).
...
> INSERT 50 {"x": 1}
OK: Record added
> INSERT 30 {"x": 2}
OK: Record added
> INSERT 70 {"x": 3}
OK: Record added
> INSERT 20 {"x": 4}
OK: Record added
> INSERT 80 {"x": 5}
OK: Record added
> SELECT 20
OK: Found {'x': 4}
> SELECT 80
OK: Found {'x': 5}
> q

```

У GIF `results/GIFs_with_database/splay_db_animation.gif` чудово видно ключову особливість `splay`-дерева: після `SELECT 20` саме вузол 20 опиняється в корені, а після `SELECT 80` - вже вузол 80. Це і є та сама "поведінка `hot-set` про яку ми писали в розділі 8.5.

11.4 Згенеровані графіки та GIF-анімації

Усі зображення з'являються автоматично після запуску `benchmark.py` у папці `results/plots/`. Для нашого основного прогону ($N = 100000, 150000$) збережено сітку 2×2 (`lines_all.png`) та окремі лінійні і стовпчикові діаграми. Усі PNG-файли виконані в єдиній темній палітрі (фон `#111318`) - так само, як і візуалізації самих дерев у GIF.

12 ПІДСУМОК

Ми реалізували повноцінну навчальну "СУБД на деревах" із шістьма самобалансованими структурами, поєднаних спільним абстрактним інтерфейсом. Кожна операція бази даних візуалізується покадрово і зберігається в GIF, а окремий бенчмарк-скрипт об'єктивно (з медіанами, фіксованою випадковістю та контролем GC) порівнює всі реалізації між собою та з SQLite. Результати показали очікувані теоретичні закономірності (логарифмічна складність на операцію), оголили цікаві практичні нюанси (сплеювання шкодить на рівномірному доступі; B+ дерево найкраще видаляє завдяки локалізації даних у листах), і дали зрозуміти, яка прірва відділяє чистий Python від оптимізованої C-СУБД на кшталт SQLite. Найважливіше - проєкт не залишився "набором дерев а став системою, у якій кожна частина (ADT-контракт, DatabaseEngine, візуалізатор, бенчмарк) виконує свою роль і ці ролі добре стикуються між собою. Саме такий підхід ми будемо використовувати в наступних, уже не навчальних, проєктах.